

Technikai interjú felkészítő

Node.js fejlesztő pozíció — várható technikai kérdések & mintaválaszok · jelmagyarázat: **ALAP** kötelező elvárás · **EXTRA** előny

A hirdetésben **konkrétan megnevezett technológiákra** fókuszálva: Node.js/TypeScript, REST & GraphQL, SQL & NoSQL, üzenetküldési minták, NATS/RabbitMQ, Kafka, event-driven architektúra, valamint .NET/C# és Azure serverless. Minden válasz úgy van megírva, hogy **kimondható** legyen ~30–60 másodperc alatt.

- | | |
|----------------------------|-------------------------------------|
| 1 Node.js futásidő & async | 8 Message brokerek: NATS & RabbitMQ |
| 2 TypeScript | 9 Event streaming: Kafka |
| 3 REST API-k | 10 Event-driven architektúra |
| 4 GraphQL | 11 Debug & teljesítmény |
| 5 Relációs adatbázisok | 12 .NET / C# |
| 6 NoSQL: MongoDB & Redis | 13 Azure Functions & serverless |
| 7 Üzenetküldési minták | 14 Mikroszolgáltatások |

1 Node.js futásidő & async **ALAP**

Magyarázd el az event loop-ot és a fázisait.

A Node egy szálon futtatja a JS-t a libuv event loopján. Minden kör rendezett fázisokon megy végig: **timers** (`setTimeout` / `setInterval`), **pending callbacks**, **poll** (I/O, itt fut a legtöbb callback), **check** (`setImmediate`), majd **close** callbackek.

Minden fázis között — és minden makrotask között — kiürülnek a mikrotask sorok: előbb a `process.nextTick`, aztán a feloldott Promise-ok. Emiatt egy `nextTick` ki tudja éhezteni a loopot, ha folyamatosan újra beütemezi magát.

`setImmediate` VS `setTimeout(fn, 0)` VS `process.nextTick` ?

- `process.nextTick` — a loop folytatása előtt fut, legmagasabb prioritás, ugyanabban a körben.
- `setImmediate` — a **check** fázisban fut, a poll/I/O befejezése után.
- `setTimeout(fn, 0)` — a **timers** fázisban; minimum ~1ms, a sorrend az immediate-hez képest a fő modulból nem determinisztikus, de I/O callbacken belül mindig az immediate nyer.

A Node egy szálon fut — hogyan kezeli a nehéz feladatokat?

Az I/O-t a libuv thread pooljába szervezi ki (alapból 4, `UV_THREADPOOL_SIZE`), így az nem blokkol. De a **CPU-igényes** munka igenis blokkolja a loopot. Lehetőségek: `worker_threads` a CPU-munkára a folyamaton belül, `child_process` / `cluster` a magok közötti szétosztásra, vagy a feladatot egy sorba tolod és a fő szálon kívül dolgozod fel.

Hogyan kerülöd el a kezeletlen promise rejectiont és a callback–promise keveredést?

Inkább `async/await` + `try/catch`; a fire-and-forget promise-okra tegyél `.catch`-et. Biztonsági hálóként figyelj a `process.on('unhandledRejection')`-re, ami logol és kilép, a régi callback-alapú API-kat pedig `util.promisify`-jal csomagold be, ahelyett hogy egymásba ágyaznád őket.

2 TypeScript ALAP

`interface` vs `type` — mikor melyiket?

`interface` objektum/kontraktus alakokra, amiket bővíthetek, vagy amiknek jól jön a declaration merging (pl. az Express `Request` kiegészítése). `type` unionökre, intersectionökre, tuple-ökre, mapped és conditional típusokra — bármire, amit interface nem tud kifejezni. A gyakorlatban sima objektumoknál sokban átfednek.

Mire jók a generikusok egy valós API-ban?

Arra, hogy egy szignatúra megőrizze a hívó típusát. Egy repository `findById<T>(id): Promise<T | null>`, egy tipizált `ApiResponse<T>` boríték, vagy egy handler-wrapper, ami végigviszi a request/response body típusokat. A haszon: fordításidejű biztonság anélkül, hogy entitásonként duplikálnám a kódot.

`unknown` vs `any` ?

Az `any` kikapcsolja az ellenőrzést — csendben szétterjed. Az `unknown` a biztonságos top típus: bármit tarthatsz benne, de használat előtt szűkítened kell (`typeof`, `guard`, `séma-parse`). Én az `unknown`-t a határokon használom — JSON parse, catch ág —, majd validálok konkrét típusra.

Hogyan validálok a külső bemenetet TS-ben, ha a típusok futásidőben eltűnnek?

A típusok fordításkor törölődnek, így egy JSON body valójában nem az a típus, aminek annotálva van. A határon validálok egy séma-könyvtárral (Zod, io-ts, class-validator), és **a TS típust a sémából származtatom**, hogy a futásidejű ellenőrzés és a statikus típus ne tudjon szétcsúszni.

3 REST API-k ALAP

Mitől RESTful egy API, és hogyan választasz státuszkódot?

Erőforrás-orientált URL-ek (főnevek), HTTP igék a műveletekre, állapotmentesség, helyes státusz-szemantika. Kódok: `200/201/204` siker, `400` rossz bemenet, `401` nem hitelesített vs `403` nincs jogosultság, `404` nincs meg, `409` konfliktus, `422` validáció, `429` rate limit, `500` szerverhiba, `503` leállt függőség.

Mely igék idempotensek, és miért számít?

`GET`, `PUT`, `DELETE`, `HEAD` idempotens; a `POST` nem. Azért számít, mert a kliensek, proxyk és üzenet-fogyasztók újrapróbálkoznak — egy idempotens végpont elviseli a duplikált kézbesítést. Nem idempotens létrehozásnál idempotency key-t teszek hozzá, hogy egy újraküldött kérés ne szúrjon be kétszer.

Hogyan verziózol és lapozol egy API-t?

Verziózás: URI (`/v1/`) az egyszerűségért és a cache-barátságért, vagy header/media-type, ha tisztább URL-eket akarok. Lapozás: **offset/limit** kis adathalmazra és oldalra ugráshoz; **cursor/keyset** nagy vagy élő adatra, mert az offset lassul és sorokat ugorhat át, ha az elemek mozognak.

Hogyan védesz meg egy REST végpontot?

Hitelesítés JWT-vel vagy opaque tokennel (rövid életű access + refresh), jogosultság-ellenőrzés erőforrásonként, minden bemenet validálása, rate limit, explicit CORS, és sose bízom a kliens által küldött ID-kben a tulajdonlásnál. A titkok vaultban/env-ben vannak, nem a kódban. HTTPS végponttól végpontig.

4 GraphQL ALAP

Mikor jobb a GraphQL, mint a REST?

Amikor a klienseknek eltérő adat-alakokra van szükségük és el akarod kerülni az over/under-fetchinget, vagy amikor egy képernyő sok erőforrást aggregál — a GraphQL ezt egyetlen tipizált kérésbe sűríti. A REST egyszerűbb marad sima CRUD-hoz, fájlküldéshez és HTTP-szintű cache-eléshez.

Magyarázd el az N+1 problémát és a megoldását.

Lefut egy lista-resolver, aztán egy beágyazott mező resolvere elemenként egyszer elsül — 1 lekérdezésből N+1 lesz. Megoldás a **DataLoader**: az egy körben összegyűjtött elemenkénti kulcsokat egyetlen backend-hívásba batcheli, és kérésenként cache-el. Emellett query mélységet/komplexitást is korlátozok, hogy a kliens ne kérhessen le drága beágyazott robbanást.

Query vs mutation vs subscription?

Query olvas (a mezők párhuzamosan resolve-olnak). **Mutation** ír (a top-level mezők sorosan futnak, így a sorrend definiált). **Subscription** frissítéseket push-ol egy tartós kapcsolaton — általában WebSocketen, gyakran egy pub/sub broker mögötte.

5 Relációs adatbázisok ALAP

Vedd végig az izolációs szinteket.

- **Read Uncommitted** — dirty read lehetséges.
- **Read Committed** — nincs dirty read (Postgres alapértelmezett).
- **Repeatable Read** — ugyanazok a sorok ugyanúgy olvashatók újra; blokkolja a non-repeatable readet.
- **Serializable** — teljes izoláció, mintha a tranzakciók egyesével futnának.

Magasabb izoláció = kevesebb anomália, de több versengés. A legalacsonyabb szintet választom, ami az adott műveletnél még helyes.

Hogyan diagnosztizálsz és javítasz egy lassú lekérdezést?

`EXPLAIN ANALYZE` -zal kezdek, hogy lássam a valódi tervet — sequential scan nagy táblán, rossz sorbecslés, drága rendezés/join a gyanús. Javítás: index hozzáadása/javítása (composite a jó oszlopsorrendben, covering index), lekérdezés átírása, statisztikák frissítése, vagy egy forró olvasási út denormalizálása.

Mikor *nem* segít — vagy árt — az index?

Nem segít alacsony szelektivitású oszlopon (egy boolean), vagy ha a lekérdezés függvénybe csomagolja az oszlopot, hacsak nincs rá illeszkedő function index. Árt az írási átbocsátásnak és a tárhelynek, és túl sok index lassítja az insert/update-et. A leftmost-prefix szabály miatt egy composite index csak azokat a lekérdezéseket szolgálja ki, amik a vezető oszlopait használják.

PostgreSQL vs SQL Server — mire figyelsz?

Mindkettő stabil relációs motor; az alapok átvihetők. A különbségek főleg dialektus és feature szinten vannak: azonosító-idézés, `LIMIT` vs `TOP / OFFSET FETCH`, procedurális nyelv (PL/pgSQL vs T-SQL), és a Postgres extrái, mint a gazdag JSONB és tömb típusok. Hordozható SQL-t írok, a többit az ORM/driver intézi.

6 NoSQL: MongoDB & Redis ALAP

Mikor választasz MongoDB-t relációs DB helyett?

Amikor az adat dokumentum-alakú, a séma fejlődik, és az elérési minták előre ismertek, így a dokumentumokat az olvasások köré tervezhetem. Rugalmas/beágyazott adatban és vízszintes shardingban erős. Relációs maradok, ha több entitáson átívelő joinra, erős keresztábra-megkötésekre és erős tranzakciós integritásra van szükség.

Beágyazás vs hivatkozás Mongóban?

Beágyazás együtt olvasott és a szülő által birtokolt adatra (rendelés + tételek) — egy olvasás, atomi update.

Hivatkozás önállóan lekérdezett vagy korlátlan/nagy aldata esetén, vagy many-to-many-nél, hogy elkerüld a hatalmas dokumentumokat és a 16MB-os limitet. Ez olvasási-minta döntés, nem normalizálási dogma.

Mire használod ténylegesen a Redist?

Cache-elés (TTL-ekkel), session tárolás, rate limiting számlálókkal, elosztott lockok, ranglisták sorted settel, és könnyű pub/sub. In-memory adatstruktúra-tároló, így ez a befutó bármihez, ami forró úton van és efemer.

Gyakori cache-elési buktatók?

Stampede — sok találat-hiány egyszerre veri a DB-t, amikor egy kulcs lejár (lockkal/jitterrel enyhíthető). **Elavult adat** — válassz stratégiát: TTL, vagy invalidálás íráskor. És a cache nem tartós, szóval sose kezeld az igazság forrásaként.

7 Üzenetküldési minták ALAP

Magyarázd el a pub/sub, request/reply, queue routing és topic mintákat.

- **Pub/Sub** — a publisher kibocsát, az adott subjectre feliratkozott összes subscriber kap egy másolatot; a publisher nem ismeri a fogyasztókat.
- **Request/Reply** — a küldő megad egy válaszcímet és vár a válaszra; RPC-szerű a brokeren keresztül.
- **Queue routing / work queue** — versengő fogyasztók egy soron, minden üzenet pontosan egy workerhez megy → terheléelosztás.
- **Topic** — minta/subject-hierarchia szerinti routing, hogy a subscriberek arra szűrjenek, ami érdekli őket.

at-most-once vs at-least-once vs exactly-once?

At-most-once: nincs újraküldés, üzenet elveszhet. **At-least-once**: újraküld ackig, így vannak duplikátumok — ez a gyakori alapértelmezés. Az **exactly-once** végponttól végpontig nehéz/drága; a gyakorlatban at-least-once + **idempotens fogyasztók** adják ki ugyanazt a hatást.

Hogyan teszel idempotenssé egy fogyasztót?

Adj minden üzenetnek stabil ID-t és tartsd nyilván a feldolgozott ID-ket (dedupe tábla vagy Redis set), vagy tervezd az írást eleve idempotensre (upsert kulcs szerint, feltételes update). Így egy újraküldés no-op lesz double-apply helyett.

Mi az a dead-letter queue?

Egy mellék-sor azoknak az üzeneteknek, amik ismételt elbuknak vagy túllépik a retry/TTL limitet. Megakadályozza, hogy egy poison message blokkolja a sort, és megőrzi vizsgálatra, riasztásra és kézi újrajátszásra ahelyett, hogy csendben eldobná.

8 Message brokerek: NATS & RabbitMQ ALAP

Hogyan routol a RabbitMQ?

A producerek egy **exchange**-re publikálnak, nem sorba. Az exchange típusa dönti el a routingot: **direct** (pontos routing key), **topic** (wildcard minták), **fanout** (broadcast minden kötött sorra), **headers**. A bindingok kötik az exchange-eket a sorokhoz; a fogyasztók a sorokból olvasnak és ackelnek.

NATS core vs JetStream?

A **core NATS** fire-and-forget pub/sub — rendkívül gyors, at-most-once, nincs tárolás. A **JetStream** perzisztenciát, at-least-once kézbesítést, replayt és fogyasztó-állapotot ad rá. Szóval core az efemer jelzésekhez, JetStream, ha tartósság vagy garantált kézbesítés kell.

RabbitMQ vagy NATS — hogyan választanál?

NATS: nagyon alacsony késleltetés, egyszerű üzemeltetés, remek nagy átbocsátású belső üzenetküldéshez és request/reply-hoz. RabbitMQ: érett, gazdag routing, üzenetenkénti TTL, prioritások, jól értett DLQ-kezelés. Az igényekhez igazítanám — könnyű+gyors a NATS felé, komplex routing/enterprise feature a RabbitMQ felé húz. A hirdetés mindkettőt használja, ami gyakori.

Hogyan kerülöd el az üzenetvesztést RabbitMQ-ban?

Durable sorok + persistent üzenetek, **publisher confirm**, hogy a broker visszaigazolja a megérkezést, kézi fogyasztó-ack csak sikeres feldolgozás után (**prefetch** -csele a folyamatban lévő munka korlátozásához), és DLQ a hibákra. A perzisztencia egy kis átbocsátást cserél a biztonságért.

9 Event streaming: Kafka ALAP

Miben más a Kafka, mint egy hagyományos üzenetsor?

A Kafka egy tartós, újrajátszható **log**, nem egy sor, ami fogyasztáskor törli az üzenetet. Az üzenetek megőrzési szabály szerint perzisztálódnak, és a fogyasztók a saját **offset**-jüket követik, így több consumer group függetlenül olvassa ugyanazt a streamet, és vissza is tekerhetsz. Emiatt ideális event sourcinghoz és multi-consumer fan-outhoz.

Magyarázd el a partíciókat, consumer groupokat és a sorrendet.

Egy topic **partíciókra** oszlik — ez a párhuzamosság egysége. Egy consumer groupon belül minden partíciót egy fogyasztó birtokol, így a fogyasztók hozzáadása a partíciók számaig skálázódik. A sorrend **csak partíción belül** garantált, ezért a sorrendben tartandó üzeneteket (pl. entitás-ID szerint) ugyanarra a partícióra kulcsolom.

Hogyan hatnak az offsetek és a commit stratégia a kézbesítési garanciára?

Commit a feldolgozás **után** → at-least-once (crash commit előtt ⇒ újrafeldolgozás). Commit **előtte** → at-most-once (kihagyhat). Az auto-commit kényelmes, de eldobhat vagy duplázhat az időzítéstől függően, ezért a helyességért kézzel commitolok a munka sikere után, és a fogyasztót idempotensen tartom.

Mi a consumer lag és miért figyeled?

A lag = legutolsó offset mínusz commitolt offset — mennyire van lemaradva egy fogyasztó. A növekvő lag azt jelenti, hogy a fogyasztók nem tudnak lépést tartani a producerekkel; ez a kulcs health metrika.

Partíciók/fogyasztók hozzáadásával, gyorsabb feldolgozással vagy batcheléssel kezelem.

10 Event-driven architektúra ALAP

Milyen kompromisszumokkal jár az event-driven megközelítés?

Előnyök: laza csatolás, független skálázás, ellenállóság (egy leállt fogyasztó nem blokkolja a producereket), természetes audit nyom. **Költségek:** eventual consistency, nehezebb debug az async ugrásokon át, üzenet-sorrend/duplikáció, és több mozgó infrastruktúra. Jó szétcsatolt doménekhez; egy egyszerű CRUD app-hoz túlzás.

Miért fontosak az üzenet-sémák és -kontrakthusok?

A producerek és fogyasztók időben és deployban szét vannak csatolva, így az üzenet **maga** a kontraktus. Egy séma (JSON Schema, Avro, Protobuf) plusz egy registry kikényszeríti a struktúrát, és lehetővé teszi a biztonságos fejlődést — additív/visszafelé kompatibilis változásokkal, hogy a régi fogyasztók ne törjenek el. Az event verziózása a tervezés része, nem utólagos gondolat.

Event sourcing vs event-driven — ugyanaz?

Nem. Az **event-driven** = a komponensek eventeken keresztül kommunikálnak. Az **event sourcing** = az állapotot eventek append-only sorozataként tárolod, és replayjel építed újra, ahelyett hogy a jelenlegi állapotot közvetlenül tárolnád. Lehetsz event-driven event sourcing nélkül; az event sourcing gyakran CQRS-sel párosul az olvasások kiszolgálására.

Hogyan tartod konzisztensen a DB-írást és az event-publikálást?

Nem tudsz atomian írni a DB-be és a brokerbe egy tranzakcióban — ez a dual-write probléma. Az **outbox pattern** megoldja: az eventet egy outbox táblába írod ugyanabban a DB-tranzakcióban, majd egy relay (poller vagy CDC) publikálja a brokerbe. Így az event pontosan akkor megy ki, ha az adat commitolódott.

11 Debug & teljesítmény ALAP

Hogyan találsz memory leaket Node-ban?

Figyeld az RSS/heap trendet időben; ha emelkedik és sose ül vissza állandó terhelés alatt, valószínűleg leak. Készíts **heap snapshotokat** (Chrome DevTools `--inspect` -tel) időközönként és diffeld, hogy lásd, mi marad megtartva. Szokásos okok: korlátlan cache-ek/tömbök, sose eltávolított listenerek, referenciát tartó closure-ök, nem törölt timerek.

Egy production végpont hirtelen belassult — hogyan állsz neki?

Előbb metrikák/trace-ek, hogy lokalizáld: a DB, egy upstream hívás, az event loop, vagy a GC? Nézd az **event-loop laget** és a CPU-t. Gyakori okok: hiányzó index, miután nőtt az adat, egy blokkoló sync hívás, connection pool kimerülés, vagy egy N+1. Adattal izolálom a réteget, mielőtt kódot módosítanék.

Hogyan profilozol CPU-igényes Node kódot?

`node --prof` vagy a DevTools CPU profiler / flame graph a forró függvények megtalálására. Ha egy valódi CPU-feladat blokkolja a loopot, told át `worker_thread`-be vagy szervezd ki. A cél, hogy az egy fő szál szabad maradjon az I/O-ra.

12 .NET / C# EXTRA

C# `async/await` — hogyan viszonyul a Node modelljéhez?

Koncepcionálisan hasonló — nem-blokkoló `await` egy `Task`-on. A különbség: a .NET valóban többszálú thread poolal, így a párhuzamosság elsődleges, míg a Node egy szálon `async`. C#-ban figyelek a `sync-over-async` deadlockokra (`.Result` / `.Wait()`), és library kódban `ConfigureAwait(false)`-t használok.

Mi a dependency injection és hogyan használja a .NET?

A DI egy osztály függőségeit kívülről adja be, ahelyett hogy belül newolná — jobb tesztelhetőség és szétcsatolás. Az ASP.NET Core-nak beépített konténere van; a szolgáltatásokat egy élettartammal regisztrárod — **transient** (feloldásonként), **scoped** (kérésenként), **singleton** (egy példány) —, és a konstruktoron át injektálódnak.

Node felől jöve hogyan állnál neki .NET mikroszolgáltatások karbantartásának?

Az architektúra-koncepciók átvihetők — REST, DI, config, `async`, konténerek, ugyanazok az üzenet-brokerek. Beledolgoznám magam a toolingba (SDK, EF Core, projekt/solution felépítés), és az erős tipizálásra meg a keretrendszer konvencióira támaszkodnék. A hirdetés a .NET-et maintain-and-extend jelleggel keretezi mentorálással, ami illik a meglévő backend készségek átviteléhez.

13 Azure Functions & serverless EXTRA

Mi az a cold start és hogyan csökkented?

Amikor egy függvény nulláról skálázódik, a platform új példányt indít és inicializálja a futásidőt — ez az első hívás késleltetése a cold start. Csökkentés: warm/premium plan vagy előmelegített példányok, kisebb deployment/függőségek, könnyebb futásidők. A példányok melegen tartása költséget cserél késleltetésre.

Triggerek és bindingok az Azure Functionsben?

A **trigger** az, ami meghívja a függvényt — HTTP, timer, queue/Service Bus üzenet, blob, Event Hub. A **bindingok** deklaratíván kötik be a be- és kimeneteket (olvasás egy sorból, írás egy táblába), így megspórolod az SDK-boilerplate-et. Ez tisztán ráképződik a hirdetés event-driven munkájára — egy üzenet érkezik, egy függvény feldolgozza.

Mikor rossz választás a serverless?

Állandó nagy forgalmú workloadhoz (az always-on olcsóbb lehet a hívásonkéntinél), hosszan futó vagy állapotos feldolgozáshoz, cold startra érzékeny, nagyon késleltetés-érzékeny utakhoz, vagy amikor finomhangolt kontroll kell a futásidő felett. Kiváló viszont a hullámzó, event-triggerelt, függetlenül skálázódó feladatokhoz.

14 Mikroszolgáltatások **EXTRA**

Hogyan kommunikálnak a szolgáltatások, és mikor sync vs async?

Sync (REST/gRPC), amikor a hívónak azonnali válasz kell — de ez összeköti a rendelkezésre állást és hibaláncot okozhat. **Async** (broker/eventek) szétcsatoláshoz, ellenállósághoz és terhelés-kiegyenlítéshez, ha egy eventual eredmény elfogadható. A legtöbb rendszer keveri; a hirdetés kifejezetten mindkettőt akarja: szinkron (REST/GraphQL) és aszinkron (broker/Kafka).

Hogyan kezelsz több szolgáltatáson átívelő tranzakciót?

Nincs elosztott ACID tranzakció — a **saga** mintát használom: lokális tranzakciók sorozata, mindegyik eventet bocsát ki, ami a következőt indítja, **kompenzáló akciókkal** az előző lépések visszavonására hiba esetén. Orkesztrált (egy koordinátor hajtja) vagy koreografált (a szolgáltatások egymás eventjeire reagálnak).

Hogyan akadályozod meg, hogy egy hibás szolgáltatás levigye a rendszert?

Circuit breaker, hogy leállítsd egy hibázó függőség ostromlását, timeoutok és korlátozott retryk backoff + jitterrel, bulkheadek az erőforrás-poolok izolálására, és fokozatos degradáció/fallback. Async üzenetküldéssel kombinálva egy leállt fogyasztó csak annyit jelent, hogy az üzenetek felsorakoznak, nem pedig hogy a hívók hibáznak.

Hogyan követsz egy kérést sok szolgáltatáson át?

Distributed tracing — egy **correlation/trace ID** továbbítása minden ugráson át (HTTP headerek, üzenet-metaadat), hogy a logok és spanok egyetlen idővonallá álljanak össze. Valami OpenTelemetry-félével end-to-end késleltetést kapsz, és pontosan megmondható, melyik szolgáltatás lassú vagy hibázik a láncban.